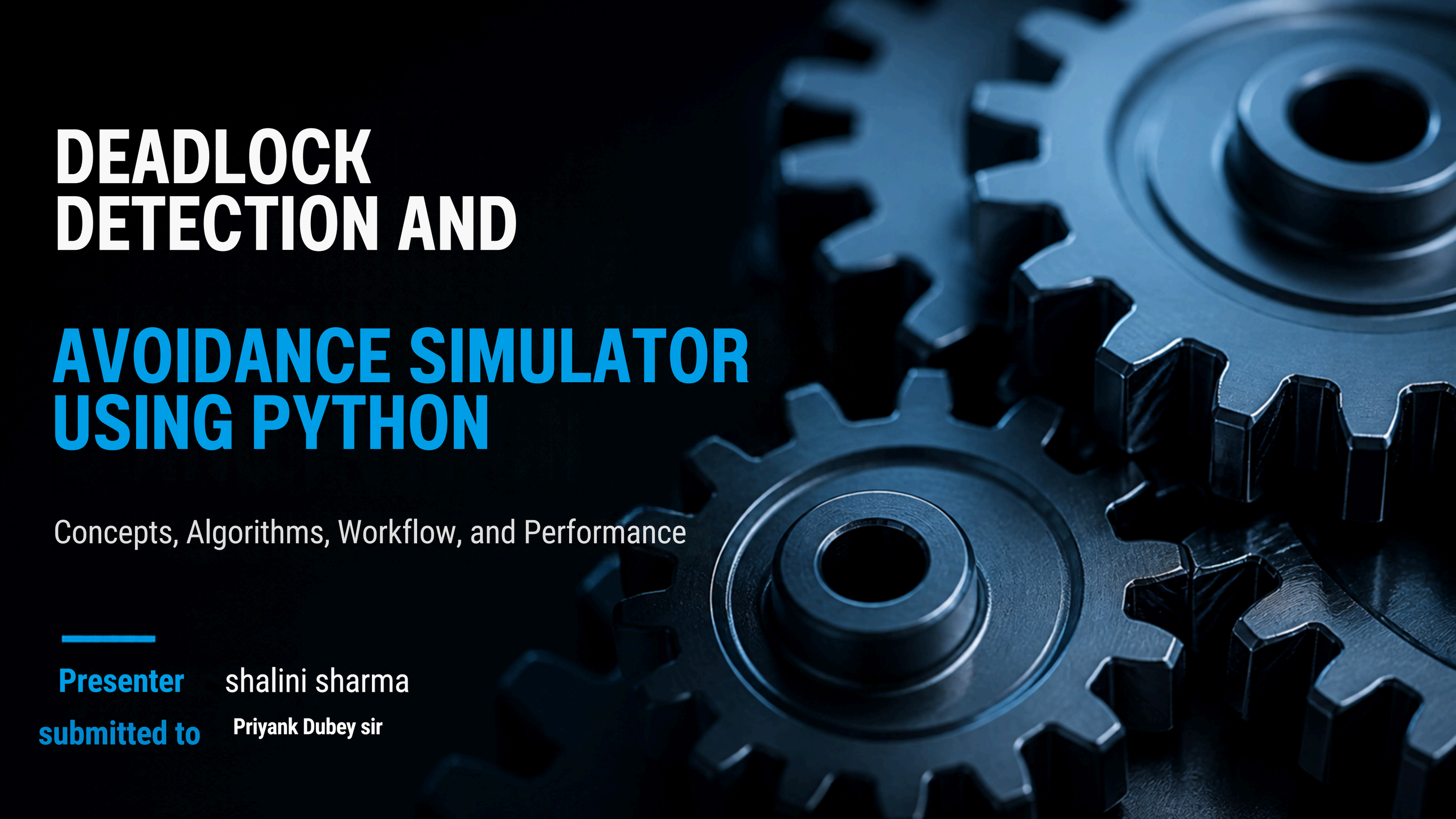
DEADLOCK DETECTION AND AVOIDANCE SIMULATOR USING PYTHON

Concepts, Algorithms, Workflow, and Performance

Presenter shalini sharma

submitted to **Priyank Dubey sir**



DEADLOCK CONCEPTS AND KEY ALGORITHMS

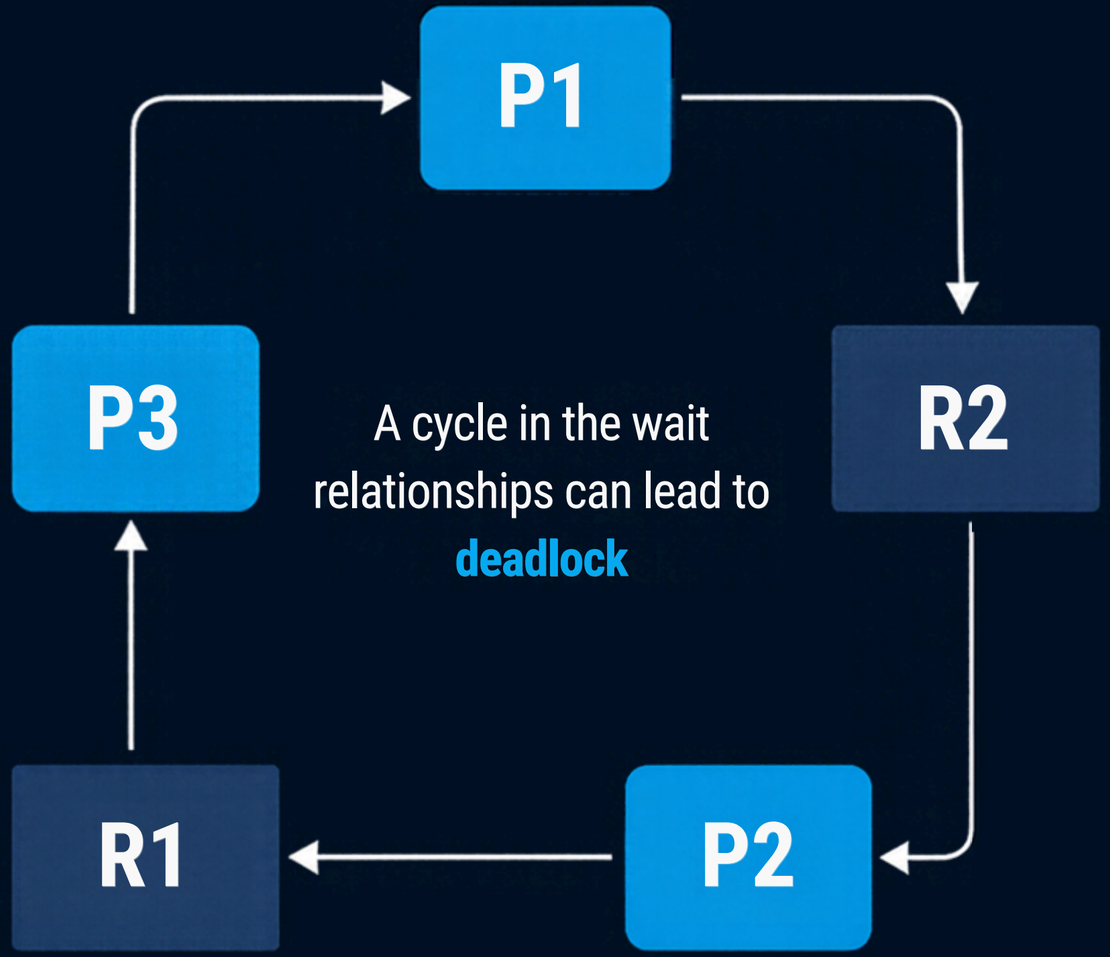
Deadlock occurs when processes wait indefinitely for resources held by others

Need matrix helps decide what a process still requires
Need = Maximum - Allocation

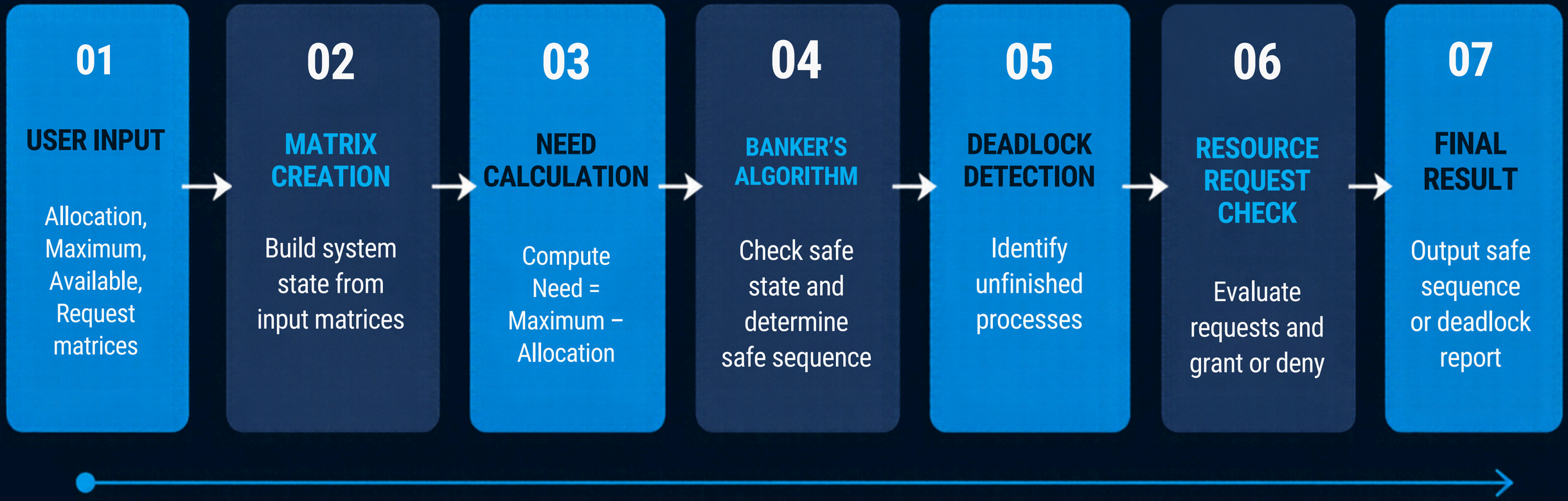
Banker's Algorithm checks if the system is in a safe state and finds a safe sequence

Sample safe sequence: P1 → P3 → P4 → P0 → P2

Deadlock Detection Algorithm identifies processes that cannot finish



SIMULATOR WORKFLOW FLOWCHART



KEY IMPLEMENTATION
FUNCTIONS AND LOGIC

calculate need()

Computes the Need matrix from Max and Allocation.

bankers Algorithm()

Checks system safety and returns a safe sequence.

detect_deadlock()

Identifies unfinished processes when no safe state exists.

resource request()

Evaluates additional requests and grants only if safe.

Clear decision points

Each function uses defined checks to ensure system safety.

```
def calculate need(Max, Allocation) :
    Need = matrix(rows, cols)
    for i in rows:
    for j in cols: Need[i][j] = Max[i][j] – Allocation[i][j]
    return Need

def bankers Algorithm(Available, Max, Allocation, Need) :
    while unfinished processes exist:
    if Need[i] <≤ Available:
    Available ±= Allocation[i]; mark i finished
    return safe sequence if all finished else failure

def detect deadlock(Available, Max, Allocation, Need) :
    if bankers Algorithm(……) == failure:
    return List of unfinished processes
    else: return none

def resource request(process, Request, Available, Max, Allocation, Need) :
    if Request <≤ Need process) and Request <≤ Available:
    pretend to allocate and run bankers Algorithm(……)
    grant if safe else rollback and deny
    else: deny request
```

SAMPLE INPUT AND OUTPUT DEMONSTRATION

INPUT

ALLOCATION (5×3)

	R1	R2	R3
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

MAXIMUM (5×3)

	R1	R2	R3
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

AVAILABLE (1×3)

R1	R2	R3
3	3	2

REQUEST FROM P1 (1×3)

R1	R2	R3
1	0	2

OUTPUT

NO DEADLOCK DETECTED

The system is in a safe state.

SAFE SEQUENCE CONFIRMED

P1 → P3 → P4 → P0 → P2

REQUEST FROM P1 GRANTED SUCCESSFULLY

PERFORMANCE ANALYSIS AND COMPLEXITY

Matrix calculation

Scales with processes and resource types

$O(n \times m)$

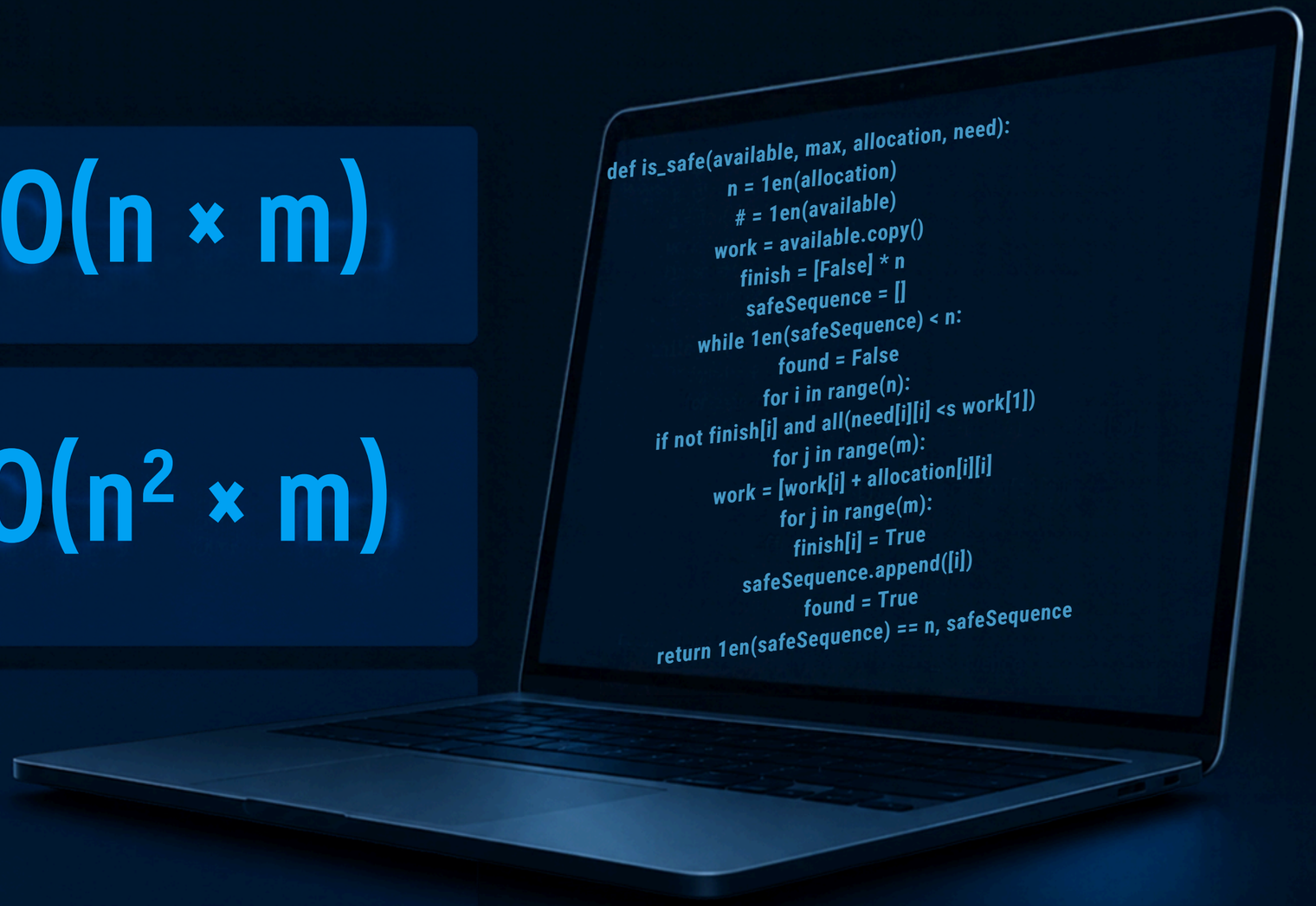
Banker's Algorithm, Deadlock Detection, Resource Request

Each involves nested checks

$O(n^2 \times m)$

What it means

Complexity grows with more processes and more resource types



CONCLUSION AND EDUCATIONAL TAKEAWAYS

The simulator **detects deadlocks** and grants requests safely.

Banker's Algorithm finds safe sequences and prevents deadlocks.

Deadlock Detection Algorithm identifies processes involved in deadlock.

Performance is **suitable for learning** with moderate-sized inputs.

Explore further **improvements** and **real-world applications.**



> Understanding and simulating **deadlock avoidance** is key to safe resource management in operating systems.